

hxp — User Manual

A live-preview workflow for **Markdown**, **LaTeX**, and **Typst**. You edit in [helix](#) — or [micro](#), via `hxp --micro` — on the left, a PDF viewer auto-reloads on the right, and every save recompiles. Compile errors are turned into a PDF (and an optional side terminal readout) so you fix and re-save without leaving the editor. Click in the PDF to jump back to the source line.

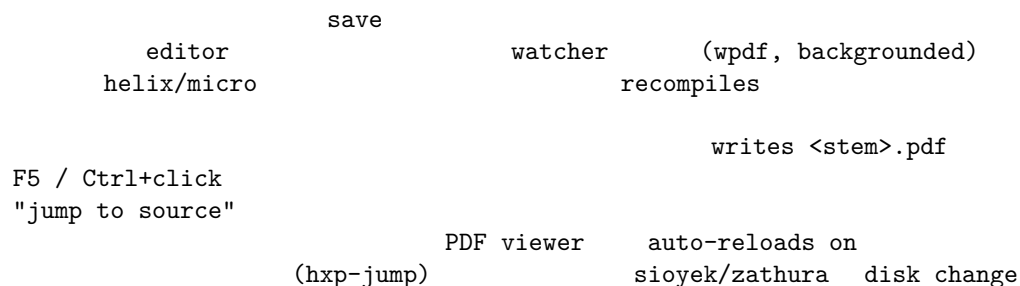
This is the operational reference. For a project overview and the install matrix, see [README.md](#).

Contents

- [Mental model](#)
- [Quick start](#)
- [Install \(recap\)](#)
- [Commands](#)
 - [hxp](#)
 - [Choosing the editor](#)
 - [hxp --doctor](#)
 - [wpdf](#)
 - [hxp_errs](#)
- [The recommended tmux layout](#)
- [Inverse search \(PDF → editor\)](#)
- [Forward search \(editor → PDF\)](#)
- [Viewer keybindings](#)
- [Per-language behavior](#)
 - [Markdown](#)
 - [LaTeX](#)
 - [Typst](#)
- [Error reporting](#)
- [Environment variables](#)
- [Window tiling](#)
- [Runtime & output files](#)
- [Troubleshooting](#)
- [Development](#)
- [Uninstall](#)

Mental model

One `hxp <file>` call starts three cooperating pieces and tears them all down when you quit the editor:



- **Editing** happens in `helix`, or in `micro` when you pass `--micro`; nothing special is required of you — just save.
- **Compiling** is done by a backgrounded watcher (`wpdf`) that fires on each save. Success replaces the PDF; failure renders an *error PDF* in its place.
- **Viewing** is a standard PDF viewer ([sioyek](#) or [zathura](#)) that reloads the file itself whenever it changes on disk.

- **Inverse search** closes the loop: clicking in the PDF (or pressing **F5**) drives your *already-running* editor to the matching source line.

When you quit the editor, **hxp** stops the watcher, closes the viewer window it opened, and sweeps its scratch files — **keeping only the finished .pdf**.

Quick start

1. Confirm the toolchain (run this first whenever something misbehaves)

```
hxp --doctor
```

2. Open or create a document — tiles editor + viewer, recompiles on save

```
hxp notes.md
```

```
hxp paper.tex
```

```
hxp slides.typ
```

If the file doesn't exist yet, hxp scaffolds a minimal starter for you.

3. Same thing in micro instead of helix

```
hxp --micro notes.md
```

For inverse search and an errors-in-your-eyeline pane, launch inside **tmux** (see [the recommended layout](#)).

Install (recap)

```
git clone https://github.com/jwKwn1028/hxp.git ~/Applications/hxp
```

```
~/Applications/hxp/install.sh
```

Then add one line to `~/.zshrc` and reload your shell:

```
[[ -r "$HOME/.zsh/hxp-main.zsh" ]] && source "$HOME/.zsh/hxp-main.zsh"
```

`install.sh` symlinks the shell functions, the `bin/` helpers, the viewer configs, and `micro`'s forward-search plugin into place (re-runnable; backs up any real files it would replace). Print the dependency list without installing via `./install.sh --deps`.

Only the editor you actually use has to be installed. Neither editor gets a keybinding from `hxp` — see [Forward search](#) for the one line to add.

PATH matters. The viewers launch `hxp-jump` (and it calls `hxp-mdline`) *by bare name*. Make sure `~/local/bin` is on **PATH before** the viewer spawns, or inverse search silently falls back to opening a fresh editor. `hxp-fwd` resolves `hxp-texline` the same way for forward search.

Commands

Three shell functions are added to your shell, plus the `hxp --doctor` diagnostic. All three accept `.md`, `.tex`, and `.typ`.

hxp <file>

The full workflow: editor + viewer + watcher, torn down together.

usage: `hxp [--micro|--helix] <file.{md,tex,typ}> | hxp --doctor`

What it does, in order:

1. **Scaffolds** the file if it doesn't exist (only for `.md` / `.tex` / `.typ`; any other missing extension is an error, not a scaffold).

2. Resolves the **project root** and therefore the real PDF path. For `.tex` and `.typ` the PDF lands next to the project root, *not* next to an included file you happened to open (see [Per-language behavior](#)).
3. Writes a **state file** under `$XDG_RUNTIME_DIR/hxp/` recording the source, PDF, viewer, editor, tmux pane, and editor window — this is what inverse search reads.
4. Runs an **initial compile** so the viewer has something to show immediately.
5. **Tiles** the editor terminal to the left half of the work area (floating WMs only — see [Window tiling](#)).
6. **Launches the viewer** on the right. For `siyok` it opens a dedicated `--new-window` and auto-enables `synctex` mode so inverse search is armed.
7. **Backgrounds the watcher** (`wpdf -q --no-initial`) and re-focuses the editor terminal.
8. **Opens the editor**. On a clean first compile it opens just your file. If the first compile *failed*, it also opens the error log (and, for Markdown, the generated `.tex`) as extra buffers so the diagnostics are right there — extra views in `helix`, extra tabs in `micro` — with the cursor already on the offending line where the compiler named one.

When the editor exits, `hxp` runs cleanup: stops the watcher, closes *its own* viewer window (via `wmctrl`, so concurrent sessions aren't disturbed), and sweeps scratch files while keeping the PDF.

Choosing the editor

`hxp` runs `helix` by default and `micro` on request. Everything else in the session is identical — same compile pipeline, same viewer, same tiling, same cleanup, both directions of `synctex` search:

```
hxp --micro notes.md           # this session only
export HXP_EDITOR=micro        # every session
hxp --helix notes.md           # override the export for one call
```

The flag just sets `HXP_EDITOR` for the invocation. Anything other than `micro` means `helix`, and `hxp --doctor` says so out loud when the variable holds something it doesn't recognise. `hxp` checks the editor's binary before doing any work, so a missing `micro` is a one-line error rather than a compiled PDF with nothing to edit it in.

Only two things actually differ, both because the editors take a cursor position in different shapes:

	helix	micro
Opening at a line	<code>hx file:line:col</code>	<code>micro +line:col file</code>
Driving a <i>running</i> instance (inverse search)	<code>:open file:line:col</code>	<code>open file</code> then <code>goto line:col</code> , via the <code>Ctrl-e</code> command bar
Forward search	<code>:sh hxp-fwd ...</code> from a keybinding	the shipped <code>hxpfd</code> <code>micro</code> plugin

Both are 1-based in line *and* column, which is what `synctex` reports, so no translation happens anywhere in between.

`hxp --doctor`

A one-screen status of every tool `hxp` can use and the feature each one enables. **Run it first when something isn't behaving.** `hxp doctor` works too. Real output from a fully-provisioned machine:

`hxp doctor`

Required

```
zsh           /usr/bin/zsh
pandoc        /usr/bin/pandoc
inotifywait   /usr/bin/inotifywait
hx            active editor (helix) — /usr/bin/hx
```

Compilers

```
latexmk       tex + md two-step synctex
xelatex       unicode/CJK pdf engine
typst         typst 0.15.0 (3ae52774)
```

Viewer

```

sioyek      active viewer

Watch / reload
watchexec   debounced watch loop

Inverse search (PDF -> editor)
hxp-jump    /home/you/.local/bin/hxp-jump
hxp-mdline  md <- tex line mapping
tmux        in-place jumps when hxp runs in tmux
xdotool     X11 keystroke fallback (no tmux)

Forward search (editor -> PDF)
hxp-fwd     /home/you/.local/bin/hxp-fwd
hxp-texline md -> tex line mapping
wmctrl      focuses the window showing this PDF
hxp-typtext typ text-search phrases (no synctex)
pdftotext   locates the page for typ text search
gdbus       pages a running zathura for typ

Window tiling
session     XDG_SESSION_TYPE=x11
wmctrl      window placement
xprop       editor window id
wm-kind     tiling

Extras
gawk        awk (mawk/busybox also work)
cjk-font    Noto Sans CJK KR

```

Marker legend: present/active · optional & absent (feature degraded, hxp still runs) · required & missing (fix this).

The *Required* editor row follows `HXP_EDITOR`: run `hxp --doctor` with it set to `micro` and the row becomes `micro`, and a `hxp fwd.lua` row appears under *Forward search* reporting whether `micro`'s plugin is installed.

wpdf <file>

Just the watcher half — recompile on save, no editor and no viewer launched. Useful when you already have an editor and viewer open, or for a headless build loop.

usage: `wpdf [-q] [--no-initial] <file.{md,tex,typ}>`

Flag	Effect
<code>-q, --quiet</code>	Suppress the per-save OK/ERR status lines.
<code>--no-initial</code>	Skip the first compile (hxp uses this; the initial frame is already rendered).

`wpdf` picks its watch strategy automatically:

- **Typst** uses native `typst watch` (incremental — much faster than full recompiles). Disable with `HXP_NO_NATIVE_TYP=1` to use the generic loop.
- **Everything else** prefers `watchexec` (250 ms debounce, handles editor swap-file churn) and falls back to `inotifywait` when `watchexec` is absent or `HXP_NO_WATCHEXEC=1` is set.

The watch is **project-wide, not just the opened file**. For `tex/typ` it tracks the *root*'s directory (recursively) and triggers on any source or bibliography change — so editing an `\input/#include`'d sibling or the `.bib` rebuilds the whole document, not only saves to the file you opened. Markdown tracks its own directory for `.md` and `.bib` changes. hxp's own scratch files (the dot-prefixed logs/intermediates and the `.hxp_build_*` dir) are excluded, so the error PDF can't trigger a recompile loop. (Native `typst watch` already tracks a `typst` document's imports on its own.)

`wpdf` sweeps its own scratch files on exit (Ctrl-C is safe — it won't take your interactive shell down with it).

`hxp_errs <file>`

A live, full-screen terminal readout of the latest compile state. Run it in a second pane next to `hxp` to keep errors in your eyeline instead of glancing at the PDF. It re-renders on every compile.

usage: `hxp_errs <file>.{md,tex,typ}>`

- **OK** state: a green check, the timestamp, and the file path.
- **ERROR** state: the error count (when > 1), the compiler's primary message, the resolved `file:line:col`, the suspect source line, and a ready-to-paste `hx <file>:<line>:<col>` jump target.

`hxp_errs` is a *reader* — it watches the log/markdown/PDF that the watcher writes. It does not compile anything itself, so always pair it with a running `hxp` or `wpdf` for the same file.

The recommended `tmux` layout

Inverse search lands *in your running editor* only when `hxp` can reach it. Neither `helix` nor `micro` speaks an editor-server protocol, so the jump is delivered by typing at the editor. The most reliable way is to launch `hxp` inside **`tmux`**: `hxp-jump` then drives the exact pane via `tmux send-keys`. Outside `tmux` it falls back to `xdotool` keystrokes (X11 only), and failing that, spawns a fresh editor.

A comfortable two-pane setup — editor+viewer on the left, error readout on the right:

```
tmux new-session \; \  
  send-keys 'hxp paper.tex' C-m \; \  
  split-window -h \; \  
  send-keys 'hxp_errs paper.tex' C-m \; \  
  select-pane -L
```

The viewer window opens as a separate GUI window (tiled right); the `hxp_errs` pane is text inside `tmux`. You end up looking at: **editor** | **error readout** in the terminal, with the **PDF** beside them.

Inverse search (PDF → editor)

Jump from a spot in the PDF to the source line that produced it.

Viewer	Trigger
sioyek	Hover the text and press F5 (one-shot; syntex mode is auto-enabled, no F4 needed). Ctrl+click also works.
zathura	Ctrl+click on the text.

Under the hood the viewer runs `hxp-jump <file>:<line>[:<col>]`, which:

1. **Maps Markdown back.** If syntex points at the generated `.../hxp_build_<stem>/<stem>.hxp.tex`, it resolves that to your original `<stem>.md` and approximates the line with the shared `hxp-mdline` heuristic.
2. **Finds the session** via the per-source state file `hxp` wrote. State from a session that has since exited (killed hard, before cleanup ran) is skipped and reaped — its pane/window id may have been recycled, so trusting it could type the jump into an unrelated window.
3. **Delivers the jump** by preference: `tmux send-keys` → `xdotool` keystrokes to the editor's X11 window → spawning a fresh editor as last resort. What gets typed depends on the editor the state file names (see below).
4. **Handles multi-file projects:** if the syntex-reported file has no exact state file (e.g. an `\input`'d chapter), it matches any live session whose source lives in the same directory tree.

Requirements for in-place jumps: working syntex data, plus either `tmux` (launch `hxp` inside it) or `xdotool` on X11. Markdown inverse search additionally needs `latexmk` — without it the Markdown path can't emit syntex at all (see below). `hxp --doctor`'s "Inverse search" section tells you which of these are live.

What gets typed

helix takes it in one command: `Escape Escape`, then `:open <file>:<line>:<col>` and `Enter`.

micro has no positional `open` — its `open` command takes a bare path and `goto` moves within the current buffer — so the jump is up to two command-bar round trips: `Ctrl-e open <file>` `Enter`, then `Ctrl-e goto <line>:<col>` `Enter`. Two details make that safe:

- **The `open` is skipped when the file is already up.** **micro**'s `open` builds a fresh buffer and the old undo history goes with it, which would be a steep price for a click that only needed to scroll. Nothing can be asked of **micro** from outside, so **hxp-jump** keeps its own note — a one-line `<sha1>.state.buf` beside the state file — of the last file it drove the session to, seeded with the file **hxp** opened. A jump into an `\input`'d chapter and back therefore opens twice and scrolls the rest of the time.
- **The leading `Escape` gets its own beat.** **micro**'s input layer waits ~50 ms after a lone `ESC` to see whether an escape sequence follows; a key delivered inside that window arrives as `Alt-<key>`, which is not the command bar — so the `open ...` text would land in your document instead. **hxp-jump** pauses between the two.

Paths are quoted the way `/bin/sh` would quote them, since that is how **micro**'s command bar parses its arguments — spaces in filenames survive.

Forward search (editor → PDF)

The other direction: move the PDF to the line under your cursor. **hxp** ships the shim, not the keybinding — bind it yourself in whichever editor you use.

helix — `~/.config/helix/config.toml`:

```
[keys.normal]
"C-l" = ":sh ~/.local/bin/hxp-fwd '${buffer_name}' ${cursor_line} ${cursor_column}"
```

micro — `~/.config/micro/bindings.json`:

```
"Ctrl-l": "command:hxp fwd"
```

micro keybindings can't interpolate the cursor position into a command, so the **hxp fwd** command comes from a small plugin `install.sh` links into `~/.config/micro/plugin/hxp fwd/`. It reads the buffer's absolute path and cursor, converts **micro**'s 0-based coordinates to **synctex**'s 1-based ones, and runs **hxp fwd** as a background job — not a blocking call, because the `typst` fallback below shells out to `pdftotext` over the whole document. Anything **hxp fwd** prints comes back in **micro**'s info bar. Note that this replaces **micro**'s default `Ctrl-l` (`goto prefill`); that command is still there as `Ctrl-e goto`.

Either way **hxp fwd** then:

1. **Reads the session state file** for the PDF **hxp** is actually producing and the viewer it launched. Both are recorded at launch precisely so this step doesn't have to guess: **sioyek** runs a single shared instance whose command line names only the first document ever opened, so sniffing the process list identifies the wrong viewer — or none — for every later document. As with inverse search, state from an exited session is skipped and reaped, and a file with no state of its own (an `\input`'d chapter) matches any live session in the same directory tree.
2. **Re-expresses the query in **synctex**'s coordinates.** This is the whole game for **Markdown**: **synctex** records only **pandoc**'s generated `.../.hxp_build_<stem>/<stem>.hxp.tex`, never your `.md`, so a query naming the `.md` matches nothing (**SyncTeX Warning: No tag for <file>.md**) and the viewer silently fails to move. **hxp-texline** — the mirror of **hxp-mdline** — maps the cursor line onto the intermediate first. `.tex` needs no translation. `.typ` has no **synctex** at all — see the fallback below.
3. **Focuses the window showing that PDF** (via `wmctrl`, matching on the filename in the title, the same rule **hxp** uses when it tiles the viewer). **hxp** opens each document with `--new-window` inside **sioyek**'s shared instance, so the instance's own notion of the "current" window belongs to whichever session you touched last — without this step a forward search can scroll a different document's window.
4. **Issues the jump:** `--forward-search-*` for **sioyek**, `--synctex-forward` for **zathura** (which reaches a running instance over `dbus`).

The typst fallback

Typst emits no syntex — 0.15.x has no source-position output of any kind — so there is nothing to query and no way to do this exactly. Instead **hxp-fwd** looks for the cursor line’s own **text** in the rendered page:

1. **hxp-typtext** reduces the line to the prose that will actually be typeset. Code, math, references and markup are cut out as *separators*, not joined over: dropping `$E = m c^2$` from `We compute $E = m c^2$ and then refine` leaves two separate runs, and their concatenation appears on no page. Each surviving run is offered as a candidate, longest first, then nearby lines. Lines that are purely code (`#let`, `#set`, `#import`, comments...) yield nothing at all.
2. **pdftotext** extracts the page text once, and the first candidate found fixes the page.
3. The viewer is moved there: **sioyek** gets `--page` (1-based) plus `--focus-text` for a visual mark on the line; **zathura** is paged over `D-Bus` (`GotoPage`, 0-based). The `D-Bus` route matters — **zathura** only forwards `--syntex-forward` to a running instance, so `--page` or `--find` would open a *second* window on the same document.

This is an approximation, and page-level at best. It only works for text that reaches the page verbatim, so ordinary prose, headings and captions are fine, while content produced by `#let` bindings, templates, `#show` rules, or math is not. Repeated text lands on its first occurrence. If **pdftotext** is missing, or nothing on or near the line renders as searchable text, forward search falls back to raising the window.

When there’s nothing to sync *and* nothing to search for — a Markdown line that can’t be located in the intermediate, or typst text that never reaches the page — it raises the window and stops, rather than sending a Markdown line number to be read as a LaTeX one and scrolling somewhere arbitrary.

hxp --doctor’s “Forward search” section reports whether **hxp-fwd**, **hxp-texline**, **hxp-typtext**, **wmctrl**, **pdftotext** and **gdbus** are live.

Viewer keybindings

sioyek (custom additions)

These are layered on top of sioyek’s stock bindings (which still work — e.g. F8 dark mode, F9 fit-width); they add zathura-style letter keys on keys sioyek left free.

Key	Action
F5	Inverse search at the cursor → jump to source in the editor
i	Toggle inverted / dark colors
a	Fit page to window width
Ctrl-d / Ctrl-u	Half-page down / up
Shift-J / Shift-K	Next / previous page (lowercase j/k stay visual-mark moves)
H / L	Top / bottom of current page
,	Go to mark
Ctrl-l	Reload document (auto-reload is also on)

The PDF **auto-reloads** on disk change (`auto_reload_preference 1`), so a successful recompile appears without any keypress. To keep stray launches (e.g. opening a PDF from a file manager) tidy, sioyek is configured single-window / single-instance; **hxp** overrides this per-session with `--new-window` so concurrent sessions on different workspaces each get their own window.

Optional — dual panel. A `_dual_panelify` command (side-by-side variant of the current PDF, via [sioyek-python-extensions](#)) is pre-wired in `prefs_user.config` but its keybinding is commented out. To enable it: create the venv and uncomment the `d` binding (see below).

```
venv="${XDG_DATA_HOME:-$HOME/.local/share}/sioyek-extensions"
python3 -m venv "$venv" && "$venv/bin/python" -m pip install sioyek
# then uncomment `_dual_panelify d` in config/sioyek/keys_user.config
```

Override the `venv` location with `HXP_SIOYEK_VENV`.

zathura

zathura keeps its **stock keybindings**. hxp only configures behavior, not keys: clipboard selection, syntex on (`Ctrl+click` → `hxp-jump`), incremental search, and open-fit-to-width. The PDF reloads automatically on disk change.

Per-language behavior

The three languages share the loop but differ in toolchain, project-root detection, and what makes inverse search work.

Markdown (.md)

Pipeline (preferred, when `latexmk` is present):

```
pandoc md → .hxp_build_<stem>/<stem>.hxp.tex    (LaTeX, in a build dir)
latexmk <stem>.hxp.tex → PDF    (-syntex=1)      (xelatex lualatex pdflatex)
```

The two-step path exists so **syntex survives** — that’s what makes Markdown search work in either direction. Syntex knows only the `.hxp.tex`, so both directions translate: `hxp-mdline` maps a tex line back to your `.md` for inverse search, and `hxp-texline` maps a `.md` line onto the tex for forward search. Without `latexmk`, hxp falls back to `pandoc` compiling straight to PDF: still renders, but **no working syntex in either direction**.

- **Reader format:** `markdown+tex_math_single_backslash` (so `\(…\)` / `\[…\]` math works).
- **Links** are colored (`colorlinks`, blue).
- **Citations.** If your YAML frontmatter declares `bibliography:`, hxp runs `--citeproc` and lets pandoc load it (it will *not* also pass `--bibliography`, which would double-load and duplicate entries). Otherwise, if a single sibling `.bib` exists next to the source, it’s auto-passed as `--citeproc --bibliography=<that.bib>`.
- **Margins.** Pandoc’s LaTeX template only loads the `geometry` package when a `geometry` variable is set, so Markdown would otherwise inherit `article`’s very wide defaults (~1.85in). hxp passes `0.75in` on all four sides. Override with `HXP_MD_MARGIN` — a bare length (`HXP_MD_MARGIN=2cm`) applies to all sides, while anything containing `=` goes to the `geometry` package verbatim (`HXP_MD_MARGIN="top=2cm,left=3cm"`). `HXP_MD_MARGIN=` (empty) opts out and restores the class defaults. If your frontmatter sets `geometry:`, hxp leaves it alone — the document always wins.
- **CJK.** For Korean/Japanese/Chinese glyphs, hxp auto-selects a CJK font for the `xelatex/lualatex` engine (so they render instead of vanishing). It probes, in order: *Noto Sans CJK KR*, *Noto Serif CJK KR*, *NanumGothic*, *NanumMyeongjo*, *Baekmuk Batang*. Override with `HXP_CJK_FONT="Your Font"`. If your frontmatter already sets `CJKmainfont:`, hxp leaves it alone.

LaTeX (.tex)

```
latexmk -pdf -syntex=1 <root.tex>    (pdflatex; -bibtex if a sibling .bib exists)
```

- **Engine:** plain `latexmk -pdf`, i.e. `pdflatex`. If you need `xelatex/lualatex` for a `.tex` document (Unicode, `fontspec`, ...), configure it yourself — a `latexmkrc`, or a `% !TEX program = xelatex` line your `latexmk` respects. (The `xelatex` engine hxp auto-selects applies to the *Markdown* path and to error PDFs, not to `.tex`.)
- **Project root.** If the file you open contains `\documentclass`, it *is* the root. Otherwise hxp walks up the directory tree looking for a `main.tex`, `root.tex`, or `thesis.tex` that has a `\documentclass`, and compiles that — so you can open and edit an `\input`’d chapter and still get the whole document’s PDF. The PDF and syntex sidebar live next to the root.
- **Live on any project file.** The watch covers the root’s whole directory tree, so saving *any* `\input`/`\include`’d chapter (or a `.sty/.cls`) rebuilds the PDF — not just the file you opened with hxp.
- **Bibliography.** A sibling `.bib` enables `-bibtex` automatically, and editing it re-runs the build.

Typst (.typ)

```
typst watch --root <root_dir> <root.typ> <pdf>    (incremental; the default)
typst compile --root <root_dir> <root.typ> <pdf>    (one-shot / generic loop)
```


- **Native watch.** Typst’s own incremental `typst watch` drives the loop by default — fast, and it keeps the PDF current. `hxp` consumes its status stream so that **compile errors also surface** as an error PDF + `hxp_errs` flip (previously typst errors were silent and the viewer kept the last good PDF). Set `HXP_NO_NATIVE_TYP=1` to use the generic full-recompile loop instead.
- **Project root.** `hxp` walks up for a `main.typ` (or a `typst.toml` marking a package root), and uses `--root` so cross-file `#imports` resolve. The PDF lands next to the root.
- **No syntex.** Typst doesn’t emit syntex, so **inverse search isn’t available for .typ**, and forward search can only approximate: it finds the cursor line’s text in the rendered page and jumps to that page (see [the typst fallback](#)). Everything else (live reload, error PDFs, the `hxp_errs` readout) works.
- **No LaTeX needed.** Typst error PDFs are rendered *with typst itself*, so `.typ` users don’t need a LaTeX toolchain at all.

Error reporting

A failed compile is uniform across all three languages: `hxp` renders an **error PDF** in place of the document and flips `hxp_errs` to **ERROR**. The viewer keeps showing a PDF — now the error one — so you never stare at a stale page wondering if the save took.

The error PDF leads with a red “**Compile failed**” banner (with an approximate error count when more than one), then:

- **Look Here First** — the compiler’s primary message, the resolved `file:line:col` (as a clickable link), and a copy-paste **editor target** shaped for the editor the session is running (`hx file:line / micro +line file`).
- **Suspect Line** — the exact source line the error points at.
- **Nearby Source** — a few lines of context (for `.tex / .typ`).
- **Compiler Extract** — a windowed slice of the raw log (capped so a cascading LaTeX failure can’t produce a 50-page error PDF).
- **(Markdown only) “Where this really is”** — because pandoc errors reference the *generated* LaTeX, `hxp` shows the accurate `.tex` line **and** a heuristic best-guess at the corresponding Markdown line.

Notes:

- **Typst** halts at the first error by default (so you usually fix one, save, repeat). **pandoc/LaTeX** can cascade, hence the multi-error count and the windowed extract.
- Error PDFs use a **plain hyphen** in the header and conservative packages so they compile even on a pdflatex-only setup.

Environment variables

Variable	Effect
<code>HXP_EDITOR</code>	<code>helix</code> (default) or <code>micro</code> ; anything else is treated as <code>helix</code> . <code>hxp --micro / hxp --helix</code> set it for one invocation.
<code>HXP_VIEWER</code>	Force <code>sioyek</code> or <code>zathura</code> instead of auto-detect (<code>sioyek</code> preferred when both present).
<code>HXP_CJK_FONT</code>	Override the CJK font family for Markdown PDFs.
<code>HXP_MD_MARGIN</code>	Page margins for Markdown PDFs (default <code>0.75in</code>). Bare length = all sides; contains = $\rightarrow$ passed to <code>geometry</code> verbatim; empty = LaTeX class defaults. Frontmatter <code>geometry</code> : overrides it.
<code>HXP_NO_NATIVE_TYP=1</code>	Use the generic compile loop instead of <code>typst watch</code> (full recompiles; same error surfacing).
<code>HXP_NO_WATCHEXEC=1</code>	Use <code>inotifywait</code> instead of <code>watchexec</code> for the generic loop.
<code>HXP_NO_TILE=1</code>	Disable <code>wmctrl</code> tiling of the editor / viewer windows.

Variable	Effect
HXP_WM	Force tiling or floating instead of auto-detect (see below).
HXP_SIOYEK_VENV	Path to the sioyek-python-extensions venv (default <code>\${XDG_DATA_HOME:-~/local/share}/sioyek-extensions</code>).
HXP_AWK	Force a specific awk binary for log parsing (mainly for the tests).

Set inline for one run (`HXP_VIEWER=zathura hxp notes.md`) or export from `~/.zshrc` to make it your default.

Internal (set by hxp, not for you): HXP_VIEWER_PID, HXP_VIEWER_KIND.

Window tiling

Auto split-screen (editor left, viewer right) needs **X11** plus `wmctrl` and `xprop`. Check your session:

```
echo "$XDG_SESSION_TYPE" # x11 or wayland
```

- **Wayland / no wmctrl:** the tiling code silently no-ops. `hxp` still works — viewer launches, watcher recompiles, error PDFs render, inverse search works. You just place the windows yourself. `HXP_NO_TILE=1` skips it explicitly.
- **Floating WMs** (xfwm, mutter-on-X11, kwin, openbox, ...): `hxp` uses `wmctrl -e` to put the editor on the left half and the viewer on the right half of the active monitor's work area.
- **Tiling WMs** (i3, sway, bspwm, dwm, awesome, xmonad, qtile, herbstluftwm, river, hyprland): geometry calls are **skipped** — managed containers ignore `_NET_MOVERESIZE_WINDOW` anyway. The viewer opens while the editor is focused and the WM's own logic (e.g. [autotiling](#) on i3) splits it as a sibling.

Detection is automatic (`wmctrl -m`, falling back to `_NET_WM_NAME`). Override with `HXP_WM=tiling` or `HXP_WM=floating` if the guess is wrong.

Runtime & output files

Created next to your source during a session. **All are swept on exit except the rendered .pdf**, which is kept.

Path	Role
<stem>.pdf	The output — kept.
<src-dir>/.<stem>.error.log	Raw compiler stdout/stderr.
<src-dir>/.<stem>.error.md	Markdown that gets rendered into the error PDF.
<src-dir>/.<stem>.debug.tex	pandoc's md→LaTeX output (Markdown path / failure fallback).
<src-dir>/.<stem>.tmp.pdf	Staging path before the atomic move to the real PDF.
<src-dir>/hxp_build_<stem>/	latexmk build tree (<code>.tex</code> ; and Markdown's <code>synctex</code> intermediate <code><stem>.hxp.tex</code>).
<pdf-dir>/<stem>.synctex.gz	Synctex sidecar (kept only while viewing).
<code>\${XDG_RUNTIME_DIR:-/tmp}/hxp/<sha1>.state</code>	Per-source state file: source, PDF, viewer, editor, tmux pane, editor window id, pid. Read by <code>hxp-jump</code> (inverse search) and <code>hxp-fwd</code> (forward search).
<code>\${XDG_RUNTIME_DIR:-/tmp}/hxp/<sha1>.state.buf</code>	micro only: the file <code>hxp-jump</code> last drove that session to, so a jump into a buffer already on screen scrolls instead of reopening. Written by <code>hxp-jump</code> , not by <code>hxp</code> .

The leading dots keep the scratch files out of `ls` and most file pickers. If a crash ever leaves them behind, they're safe to delete — the next `hxp` run recreates whatever it needs.

Troubleshooting

Start with **hxp --doctor** — most problems are a missing optional tool, and doctor names the exact feature each one gates.

Symptom	Likely cause & fix
PDF doesn't update on save	Watcher or viewer reload stalled. Confirm inotify-tools (and ideally watchexec) via doctor. On a network/odd filesystem, try HXP_NO_WATCHEXEC=1 .
Inverse search opens a <i>new</i> editor instead of jumping	hxp can't reach the running one. Launch hxp inside tmux , or install xdotool (X11). Also confirm ~/local/bin is on PATH <i>before</i> the viewer starts, so hxp-jump resolves.
Inverse search does nothing in a .md	Markdown syntex needs latexmk . Without it the Markdown path can't emit syntex. (.typ has no syntex at all — that's expected.)
Windows don't tile	You're on Wayland, or missing wmctrl/xprop , or on a tiling WM (it defers to the WM). See Window tiling ; override with HXP_WM .
CJK glyphs missing in a Markdown PDF	Install fonts-noto-cjk or set HXP_CJK_FONT="Your Font" . Confirm the resolved font in doctor's <i>Extras</i> row.
Markdown PDF margins are wrong	Set HXP_MD_MARGIN (e.g. 0.5in), or put geometry: in the document's frontmatter for a per-file value. If margins ignore both, check that geometry: isn't set in frontmatter — the document wins over the env var.
.tex Unicode/fontspec fails under pdflatex	.tex compiles with latexmk -pdf (pdflatex). Configure xelatex/lualatex yourself via latexmkrc or a % !TEX program directive.
Bibliography entries appear twice	Don't declare bibliography: in YAML <i>and</i> rely on a sibling .bib — pick one. With the YAML key present, hxp uses --citeproc alone.
Typst errors seem silent	They shouldn't be — the error PDF + hxp_errs flip should fire. If you set HXP_NO_NATIVE_TYP=1 , the generic loop is in play; either way errors surface.
Wrong viewer launches	Set HXP_VIEWER=sioyek or HXP_VIEWER=zathura .
hxp --micro still opens helix	The flag sets HXP_EDITOR for that call only, so check nothing later in the command line overrides it, and run hxp --doctor — its <i>Required</i> row names the active editor.
Ctrl-l does nothing in micro	The binding isn't hxp 's to add: put "Ctrl-l": "command:hxpfd" in ~/config/micro/bindings.json . If it reports an unknown command, the plugin isn't linked — re-run install.sh and check doctor's hxpfd.lua row.
A jump into micro typed open ... into the document	The command bar didn't open, so the text went to the buffer. Undo, and check that nothing is intercepting Ctrl-e (micro's CommandMode) — Ctrl-e showkey Ctrl-e reports what it's bound to.
Second hxp hijacks the first's window	Shouldn't happen — sioyek sessions get --new-window and are matched by PDF basename. If it does, check you're not forcing single-window via a custom sioyek config.
Scratch .<stem>.* files left behind after a crash	Safe to delete; the next run recreates them.

Development

```
zsh test/parse-test.zsh  # compiler-log parsers — needs no compilers (fast gate)
zsh test/smoke.zsh       # real compile per language (+ typst watch); skips absent tools
```

`parse-test.zsh` runs in CI under both `gawk` and `mawk` to keep the `awk` log parsers POSIX-clean. `smoke.zsh` exercises the real pipeline and skips any language whose tools aren't installed, so it's useful with a partial toolchain. Both run on every push via `.github/workflows/ci.yml`.

Source map (see [README.md](#) for the full tree):

File	Role
<code>zsh/hxp-main.zsh</code>	<code>hxp()</code> / <code>wpdf()</code> entrypoints, window tiling, viewer launch.
<code>zsh/hxp-lib.zsh</code>	Compile helpers, error rendering, <code>hxp_errs</code> , <code>--doctor</code> . Sourced by the watcher too.
<code>bin/hxp-compile</code>	Thin per-save wrapper <code>watchexec</code> calls (avoids re-sourcing <code>.zshrc</code>).
<code>bin/hxp-jump</code>	Syntex inverse-search shim (<code>tmux</code> / <code>xdotool</code> / fresh editor).
<code>bin/hxp-mdline</code>	Shared <code>md←tex</code> line-mapping heuristic (used by <code>jump</code> and error renderer).
<code>bin/hxp-fwd</code>	Syntex forward-search shim (editor → PDF), incl. the <code>typst</code> text fallback.
<code>bin/hxp-texline</code>	Shared <code>md→tex</code> line-mapping heuristic (used by forward search).
<code>bin/hxp-typtext</code>	Reduces a <code>.typ</code> line to searchable page text (<code>typst</code> has no syntex).
<code>bin/hxp-dual-panelify</code>	PATH-resolved wrapper for <code>sioyek</code> 's dual-panel extension.
<code>config/micro/plugin/hxpfwd/</code>	<code>micro</code> plugin exposing the <code>hxpfwd</code> command — <code>micro</code> keybindings can't pass the cursor position to a shell command.

Uninstall

The installer only creates symlinks; remove them and the source line.

```
rm -rf ~/.zsh/hxp-main.zsh ~/.zsh/hxp-lib.zsh \
    ~/.local/bin/hxp-compile ~/.local/bin/hxp-jump \
    ~/.local/bin/hxp-mdline ~/.local/bin/hxp-fwd \
    ~/.local/bin/hxp-texline ~/.local/bin/hxp-typtext \
    ~/.local/bin/hxp-dual-panelify \
    ~/.config/zathura/zathurarc \
    ~/.config/sioyek/prefs_user.config ~/.config/sioyek/keys_user.config \
    ~/.config/micro/plugin/hxpfwd
```

Then delete the `source "$HOME/.zsh/hxp-main.zsh"` line from `~/.zshrc`. Any `*.bak.<timestamp>` files the installer made are your originals — restore or discard them as you like.